

- Can we wait until the user finishes typing the whole query before we flag the error and suggest replacement queries?
- What is the minimum tolerable latency for the spell-checker model to provide the output so that the user doesn't see any visible lag in his/her interactions with the search engine?

Once we get clarity on these questions, we need to think about the ways to solve this problem. For any ML/NLP system design question, before we start jumping into a machine learning (ML) model, we should first ask ourselves the question — how can this problem be solved without ML? And second, does this problem need ML at all? We need to understand whether the problem exists in the current environment and, if so, how it is getting handled. In this specific example, let us consider how we can solve this problem without ML.

We can use word spelling error detection using a dictionary and suggestion of potential corrections using the standard edit distance class of algorithms. What do you see as the problem with this approach? With a static dictionary, the universe of words we can handle is quite limited. Dictionaries typically cover standard language words and do not provide adequate coverage of various named entities that are often searched for in Web search engines.

One way to overcome this limitation is to use a frequently updated dictionary. However, given that search queries typically have a long tail, we are still likely to find a percentage of query words that are not covered by the dictionary. More importantly, any word-level dictionary may not completely serve our purpose since edit distance based spell correction recommendations do not capture the query context and hence can suggest misleading corrections. Therefore, you are satisfied at this point that a supervised machine learning-based system may be a better alternative than using a dictionary (whether static or dynamically updated), as this would enable us to learn and differentiate between unseen correctly spelled queries and unseen wrongly spelled queries.

Now you need to define the metrics for intrinsic evaluation and external evaluation of your system. While you can use precision/recall/F1-Score for intrinsic model evaluation, how can you do an extrinsic evaluation of your system when plugged into the search engine in the real world? How can you figure out whether the system caught the misspelled queries well, and whether it was flagging correctly spelled queries as misspelled queries? What are the feedback signals you can use to detect this? If the system corrected a query and the user used that correction to fetch the search results and clicked on the URLs retrieved, we can use it as a proxy/secondary signal that the correction was relevant. On the other hand, how can we figure out whether the system missed cases where it should have corrected them? Consider a sequence where two 'close' queries are issued one after another. The user does not use the search results of the first

query but uses the search results of the second one. This can be a likely candidate for the cases where the system should have corrected the misspelled query but failed to do so.

Once you and the interviewer agree on the evaluation metrics, you need to start elaborating on the system design. What kind of ML problem are you planning to model for your task? Given a user input query, you need a system that can generate a corrected query if it is misspelled. If it is not a misspelled query, it just needs to reproduce the original query. One way is to first classify a query as misspelled or not. If the query is classified as misspelled, you need to have a second ML model that generates the correct query. Another alternative is to model this directly as a sequence to sequence generation problem similar to machine translation. For correctly spelled queries, the system just reproduces the input query. For misspelled queries, it generates the corrected query. Now the question is: How would you generate samples to train your system?

You can sample actual user search query logs and generate positive samples. How would you generate negative samples for your system? Look for a sequence of two 'close' queries in succession — one query result will be discarded by the user and the successor 'close' query result will be used. The head query is likely to be the incorrect query, and the successor query is the correct query. Since the incorrect queries are likely to be a small fraction of the overall queries in user query logs, we will also need to generate synthetic incorrect queries to make our sequence to sequence model learn well. How will you generate pairs of incorrect and correct queries by automatic data augmentation?

One way is to create a mechanism that can introduce random spelling errors in correct queries. A more intelligent mechanism is to list the classes of common misspellings that humans frequently make and use them to create the misspelled queries. We will leave it to you to come up with this approach. The next step is to elaborate on the internal architecture of the sequence to sequence model. Would you use LSTMs or GRUs, or use transformers with attention? Can we use any of the pre-trained generative language models and repurpose them for this task? These are some of the questions to discuss further in our next month's column.

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! Stay healthy and stay safe. **END** 

 **By: Sandya Mannarswamy**

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning, and AI.